

Módulo 10 – Capítulo 07 – Sección 01

LLM Gateway: capa de proxy centralizada para gestionar todo el tráfico hacia modelos

El Módulo 9 de este libro estableció los principios de seguridad para sistemas de IA: defense-in-depth, control en capas, detección de prompt injection, y sanitización de outputs. El LLM Gateway es el componente de plataforma donde esos principios de seguridad se materializan como controles técnicos operativos para el tráfico de inferencia de LLMs. No es solo un proxy de conveniencia que centraliza las llamadas a APIs externas: es el punto de aplicación de políticas de seguridad, el sistema de auditoría del uso de modelos, y la capa de control que hace que los principios del Módulo 9 sean aplicados de forma sistemática a cada request que llega a un modelo de lenguaje, sin depender de que cada equipo de desarrollo los implemente individualmente.

Un LLM Gateway es un servicio de proxy inverso especializado que se interpone entre las aplicaciones clientes y los proveedores de modelos —OpenAI, Anthropic, Azure OpenAI, o modelos self-hosted en vLLM o TGI— proveyendo un punto de control centralizado para todo el tráfico de inferencia de la organización. Al igual que un API Gateway tradicional (Kong, AWS API Gateway, Apigee) gestiona el tráfico HTTP hacia microservicios, el LLM Gateway gestiona las llamadas a modelos de lenguaje añadiendo funcionalidades específicas de IA: routing inteligente entre modelos, caching semántico de respuestas similares, rate limiting por equipo o proyecto, y auditoría completa de prompts y respuestas.

El rol del gateway como **punto de aplicación de controles de seguridad** es una extensión natural de su posición arquitectónica como intermediario entre los clientes y los modelos. Desde esta posición puede implementar la **detección de prompt injection** en los requests entrantes: un clasificador especializado (entrenado o basado en heurísticas) que analiza el prompt antes de enviarlo al modelo y rechaza o alerta si detecta patrones de prompt injection directa (intentos de ignorar el system prompt) o indirecta (contenido malicioso en documentos recuperados por RAG). El gateway puede también aplicar **sanitización de outputs** antes de retornarlos a los clientes: filtrar respuestas que contengan PII no esperada, bloquear outputs que contengan información confidencial según clasificadores de contenido,

y aplicar las políticas de contenido definidas centralmente. Estos controles, desarrollados en profundidad en el Módulo 9, son implementados de forma centralizada en el gateway para que ningún equipo de desarrollo necesite reimplementarlos individualmente —y para que no puedan ignorarlos si quisieran.

Las implementaciones open source más adoptadas de LLM Gateway son **LiteLLM Proxy** — que implementa el patrón de proxy con soporte para más de 100 proveedores de LLMs bajo una interfaz unificada compatible con la API de OpenAI— y **PortKey** —que añade analíticas avanzadas y gestión de prompts. Empresas como Netflix, Uber y Stripe han documentado sus implementaciones propietarias de este patrón. La ventaja principal del LLM Gateway es operacional: en lugar de que cada aplicación gestione sus propias API keys, retry logic, fallback a modelos alternativos y logging de llamadas, toda esa lógica se centraliza en el gateway y los equipos de desarrollo solo consumen un endpoint interno unificado (<https://llm-gateway.internal/v1/chat/completions>) con credenciales internas, comportamiento consistente y visibilidad completa de su uso.

Componentes principales del LLM Gateway

- **Proxy layer:** recibe requests en formato OpenAI API-compatible ([/v1/chat/completions](#)), los transforma al formato del proveedor backend seleccionado, y normaliza la respuesta a un formato unificado; los clientes son agnósticos al proveedor real.
- **Authentication y authorization:** valida tokens JWT o API keys internas, mapea cada cliente al equipo/proyecto correspondiente para atribución de costos y aplicación de políticas de rate limiting; ningún request llega al modelo sin identificación verificada.
- **Security controls:** detección de prompt injection en requests entrantes, aplicación de políticas de contenido sobre outputs antes de retornarlos al cliente, y sanitización de PII en respuestas; implementa los controles del Módulo 9 de forma centralizada.
- **Audit logger:** registra inmutablemente cada request con timestamp, cliente, modelo usado, tokens de input/output, latencia, costo estimado, y un hash del prompt (o el prompt completo si la política de retención lo permite); el audit log es el mecanismo de investigación de incidentes.
- **Fallback handler:** en caso de error del proveedor primario (rate limit, timeout, error 5xx), reintenta automáticamente con el proveedor secundario configurado, transparentemente para el cliente; elimina la necesidad de que cada aplicación implemente su propia lógica de fallback.

Nota del Arquitecto: Sin un LLM Gateway centralizado, cada equipo gestiona su propia lógica de retry, fallback y logging de llamadas a modelos, resultando en comportamientos inconsistentes, dificultad de auditoría e imposibilidad de optimizar el costo de inferencia a nivel de organización. La adopción del gateway requiere un cambio cultural pequeño —usar el endpoint interno en lugar del endpoint externo del proveedor— con un beneficio operacional grande: visibilidad completa, controles de seguridad centralizados, y resiliencia ante fallos de proveedores.

El LLM Gateway es el punto de control central de toda la infraestructura de LLMs de la organización: el lugar donde las políticas de seguridad del Módulo 9, las políticas de governance del Capítulo 08, y los controles de costo del Capítulo 09 se aplican de forma automática y sistemática. Las siguientes secciones examinan en detalle cada una de las capacidades clave del gateway: rate limiting, routing inteligente, caching y auditoría.

Módulo 10 – Capítulo 07 – Sección 02

Rate limiting y cuotas por equipo, proyecto o usuario

El rate limiting en un LLM Gateway cumple un rol doble que no tiene equivalente exacto en API gateways convencionales: protege contra el abuso —un cliente que envía requests en loop infinito— y garantiza la equidad en el acceso a recursos escasos compartidos entre múltiples equipos. Cuando diez equipos comparten un límite corporativo de tokens-per-minute (TPM) con un proveedor como OpenAI o Anthropic, sin rate limiting interno, un equipo con un pico de tráfico puede agotar el límite global, afectando a todos los demás equipos durante el período de recuperación. El rate limiting no es, en este contexto, una restricción punitiva: es un mecanismo de distribución equitativa de capacidad compartida.

La granularidad del rate limiting debe ir de más gruesa a más fina, respetando una jerarquía que refleja la estructura organizacional. El **límite global de la organización** está definido por los contratos con los proveedores: si el contrato con OpenAI establece 500,000 TPM, ese es el límite máximo absoluto que el gateway debe respetar, con un margen de seguridad (ej. 90% del límite) para evitar throttling por variabilidad en la estimación de tokens. El **límite por equipo o Business Unit** se asigna según el presupuesto de IA aprobado para cada equipo y representa la cuota garantizada de cada uno dentro del límite global. El **límite por proyecto** permite controlar el consumo de features individuales dentro de un equipo: si el equipo de NLP tiene tres proyectos y uno de ellos empieza a consumir toda la cuota del equipo, el rate limiting por proyecto protege a los otros dos. El **límite por usuario final** es opcional pero importante en aplicaciones públicas donde usuarios individuales podrían generar patrones de uso abusivos.

La implementación técnica del rate limiting distribuido en un gateway con múltiples réplicas requiere un contador compartido externo: Redis con el algoritmo de **Token Bucket** o **Sliding Window** usando primitivas atómicas (`INCR` con `EXPIRE` , o Lua scripts para atomicidad de las operaciones compuestas). El Token Bucket permite bursts controlados hasta el tamaño del bucket, lo que es adecuado para aplicaciones con tráfico variable que necesitan absorber picos breves sin degradar la experiencia del usuario. El Sliding Window suaviza el rate limiting distribuyendo la ventana de tiempo de forma continua en lugar de en

intervalos fijos, evitando el patrón de "burst al inicio de cada ventana fija" que el Fixed Window permite.

Para LLMs específicamente, las dimensiones del rate limiting van más allá de las requests-per-minute (RPM) convencionales. Los **tokens-per-minute (TPM)** son la dimensión de costo más directa: cada token de input y output tiene un precio, y el TPM es el mejor proxy del gasto por unidad de tiempo. Los **concurrent requests** limitan el número de requests que un equipo puede tener en procesamiento simultáneo, protegiendo contra patrones de uso que saturan el backend con requests paralelos. La combinación de las tres dimensiones —RPM, TPM, concurrent requests— provee un control más completo que cualquiera de ellas por separado.

Aspectos técnicos del rate limiting para LLM Gateway

- **Dimensiones de rate limiting:** tokens-per-minute (TPM) para controlar costo, requests-per-minute (RPM) para controlar throughput, y concurrent requests para controlar saturación del backend; la combinación de las tres dimensiones es más efectiva que cada una por separado.
- **Jerarquía de límites:** límite global de organización → límite por equipo → límite por proyecto → límite por usuario; el límite más restrictivo aplicable se respeta, con headers informativos de cuánto queda disponible (`X-RateLimit-Remaining` , `X-RateLimit-Reset`).
- **Redis Token Bucket:** cada cliente tiene un bucket con capacidad máxima C; se añaden R tokens por segundo; cada request consume tokens según el número de tokens de input estimados; requests rechazados devuelven HTTP 429 con `Retry-After` header calculado.
- **Graceful degradation:** cuando un equipo alcanza su límite, el gateway puede ofrecer fallback a un modelo más económico (ej. de GPT-4o a GPT-4o-mini) en lugar de rechazar directamente, con un header informativo `X-Model-Downgraded: true` .
- **Cuotas mensuales:** además del rate limiting de velocidad (instantáneo), cuotas de gasto mensual por equipo en USD que cuando se alcanzan bloquean el acceso hasta el inicio del siguiente período o hasta aprobación manual de extensión del FinOps team.

Nota del Arquitecto: El rate limiting efectivo debe ser predecible y transparente para los equipos que lo experimentan. Un equipo que recibe un HTTP 429 inesperado sin saber cuánto de su cuota ha consumido ni cuándo se resetea no puede tomar decisiones informadas sobre cómo optimizar su uso. Los headers informativos (`x-`

`RateLimit-Limit` , `X-RateLimit-Remaining` , `X-RateLimit-Reset`) y los dashboards de consumo en tiempo real son tan importantes como el enforcement técnico del límite.

El rate limiting con granularidad por equipo, proyecto y usuario convierte el acceso a los modelos en un recurso gestionado de forma equitativa y transparente. La siguiente sección examina el routing inteligente: cómo el gateway decide, request a request, cuál modelo y proveedor sirve mejor cada petición según criterios de costo, latencia y calidad simultáneamente.

Módulo 10 – Capítulo 07 – Sección 03

Routing inteligente: redirigir peticiones al modelo correcto según latencia, costo y calidad

El routing inteligente es la capacidad que más diferencia un LLM Gateway sofisticado de un simple proxy: en lugar de enrutar todo el tráfico al mismo modelo de forma estática, el router selecciona para cada request el modelo y proveedor que mejor satisface los criterios de optimización definidos, de forma dinámica y transparente para las aplicaciones cliente. Esta capacidad convierte la complejidad de gestionar múltiples proveedores y modelos en una ventaja operacional: la organización puede optimizar simultáneamente costo, latencia y calidad sin que los equipos de desarrollo cambien una línea de código.

El router puede operar en tres modos que se pueden combinar. El **routing basado en reglas explícitas** implementa lógica de decisión configurada en YAML o JSON que mapea condiciones del request a modelos específicos: si el contexto del request supera 16k tokens, usar el modelo con mayor ventana de contexto disponible; si el proyecto tiene el flag `use_cheaper_model=true`, usar el modelo de menor costo que cumpla el SLA de calidad mínima; si el request especifica `required_latency_sla=200ms`, enrutar al proveedor con la menor latencia media medida en los últimos 60 segundos. Este modo es predecible y auditable: las decisiones de routing son deterministas y reproducibles.

El **routing basado en el estado del sistema** es más dinámico: el router mide continuamente la latencia media y la tasa de error de cada proveedor en una ventana deslizante de 60 segundos, y ajusta el routing dinámicamente para evitar proveedores que estén experimentando degradación. El patrón de **circuit breaker** es la implementación estándar: si el proveedor A tiene una tasa de error superior al 5% en los últimos 60 segundos, el circuit breaker se abre y el router evita enviarle tráfico durante el período de recuperación (típicamente 30-60 segundos), usando el proveedor B como alternativa. Cuando el período de recuperación termina, el circuit breaker pasa al estado "half-open" y envía un pequeño porcentaje del tráfico al proveedor A para verificar si se ha recuperado, antes de restaurar el routing completo.

El **routing por contenido** es el modo más avanzado y requiere un clasificador que categorice el tipo de tarea del request antes de enrutar. Código → modelo con mejor desempeño en benchmarks de código (CodeLlama, GPT-4o con system prompt especializado); razonamiento complejo → modelo con mejor desempeño en benchmarks de razonamiento (o3, Claude Opus); resumen y extracción → modelo económico que es suficientemente bueno para estas tareas (GPT-4o-mini, Claude Haiku); análisis de imágenes → modelo multimodal. El clasificador puede ser tan simple como una lista de keywords en el system prompt o tan sofisticado como un clasificador de texto entrenado sobre el historial de requests de la organización.

Las **fallback chains** son un mecanismo complementario al routing que provee resiliencia ante fallos: `[gpt-4o, claude-3-5-sonnet, gpt-4o-mini]` define la secuencia ordenada de modelos que el gateway intentará si el primero falla. El gateway reintenta con el siguiente en la cadena ante error HTTP 429 (rate limit), 503 (service unavailable), o timeout. Esta lógica de fallback, si la implementaran los equipos de desarrollo individualmente, resultaría en código redundante en decenas de aplicaciones; centralizada en el gateway, es un comportamiento gratuito que todos los equipos heredan.

Estrategias de routing inteligente

- **Rule-based routing:** tabla de reglas configurables en YAML que mapean condiciones del request (tamaño del contexto, project_id, required_latency_sla, modelo explícitamente solicitado) a modelos/proveedores específicos; determinista y auditable.
- **Latency-aware routing:** medir la latencia media de cada proveedor en ventana deslizante de 60 segundos y dirigir el tráfico al más rápido; circuit breaker que abre automáticamente cuando la tasa de error de un proveedor supera el 5%.
- **Cost-optimized routing:** estimar el costo del request ($\text{input tokens} \times \text{precio/token} + \text{output tokens estimados} \times \text{precio/token}$) y seleccionar el modelo más barato que cumpla el SLA de calidad mínima requerida por el proyecto.
- **Fallback chains:** secuencia ordenada de modelos a intentar si el primero falla: el gateway reintenta automáticamente ante error HTTP 429, 503, o timeout; la lógica de fallback es transparente para las aplicaciones cliente.
- **A/B routing para evaluación:** enviar un porcentaje configurable del tráfico (ej. 5%) a un modelo experimental y comparar las métricas de calidad, satisfacción de usuario y costo contra el modelo de producción; facilita la evaluación de nuevos modelos con tráfico real sin riesgo para el tráfico mayoritario.

Nota del Arquitecto: *El routing inteligente tiene un requisito técnico crítico: el gateway debe poder estimar el número de tokens de un request antes de enviarlo al proveedor, para calcular el costo estimado y verificar si supera los límites de rate limiting de TPM. La estimación de tokens usando el tokenizer del modelo objetivo (tiktoken para modelos OpenAI, el tokenizer de Hugging Face para modelos Anthropic) añade latencia de microsegundos pero es imprescindible para el routing cost-aware.*

El routing inteligente convierte la complejidad de gestionar múltiples proveedores y modelos en una ventaja competitiva: la organización puede optimizar simultáneamente costo, latencia y calidad sin que los equipos de desarrollo se preocupen por qué modelo está sirviendo cada request. La siguiente sección examina el caching, que es probablemente la optimización de costo más impactante disponible en el gateway para aplicaciones con patrones de uso repetitivos.

Módulo 10 – Capítulo 07 – Sección 04

Caching en el gateway: reducción de costos para peticiones repetidas

El caching de respuestas de LLMs es la optimización de costo con mejor ratio impacto/esfuerzo disponible en el stack de un LLM Gateway, y también la más fácil de implementar incorrectamente. Para aplicaciones donde los mismos prompts o prompts muy similares se envían repetidamente —generación de descripciones de productos con el mismo template, respuestas a FAQ de soporte al cliente, análisis de documentos estructurados con el mismo schema— retornar respuestas cacheadas puede reducir el costo de inferencia entre un 20% y un 90% dependiendo del ratio de cache hit. La razón del rango tan amplio es que el hit rate depende fundamentalmente del tipo de aplicación: una aplicación de FAQ interna tiene hit rates muy altos porque las preguntas son limitadas y repetitivas; una aplicación de chat abierto tiene hit rates muy bajos porque cada conversación es única.

El **exact match cache** es el mecanismo más simple y seguro: almacena la respuesta para un prompt exactamente igual usando el hash SHA256 del (model_name + system_prompt + user_message + parámetros de sampling) como clave de Redis. El costo de implementación es mínimo y el riesgo de retornar respuestas incorrectas es cero: si el hash coincide exactamente, el request es idéntico. La limitación es el hit rate bajo en aplicaciones con prompts variables incluso cuando el contenido semántico es muy similar: "resume este documento en 3 puntos" y "resume este documento en tres puntos" producen hashes completamente diferentes. El TTL del cache debe configurarse según la naturaleza del contenido: respuestas a preguntas factuales sobre productos tienen TTLs largos (días o semanas), respuestas que dependen de información en tiempo real tienen TTLs cortos (minutos o cero).

El **semantic cache** extiende el exact cache usando embeddings para detectar prompts semánticamente similares aunque no sean idénticos. El flujo es: calcular el embedding del prompt con un modelo de embedding liviano (all-MiniLM-L6-v2, text-embedding-3-small de OpenAI), buscar el nearest neighbor en la base de datos vectorial (Redis con RedisSearch, Pinecone, Weaviate), y retornar la respuesta cacheada si la similitud coseno supera el

threshold configurado (típicamente 0.95-0.98). El semantic cache puede tener hit rates significativamente más altos para aplicaciones con variaciones de formulación, pero introduce un riesgo que el exact cache no tiene: retornar la respuesta de un prompt anterior para un prompt diferente que supera el threshold de similitud pero tiene diferencias semánticas importantes. La calibración del threshold es el paso más crítico del despliegue del semantic cache: empezar conservadoramente (0.98) y bajar gradualmente mientras se valida manualmente que las respuestas retornadas son apropiadas para los prompts que disparan el cache hit.

La **invalidación del cache** es tan importante como el caching en sí. Cuando el modelo subyacente cambia de versión, todas las entradas de cache asociadas a esa versión deben invalidarse: una respuesta correcta para GPT-4o-2024-08-06 puede ser incorrecta para GPT-4o-2024-11-20 si el modelo fue actualizado. El diseño de la clave de cache debe incluir la versión del modelo como componente explícito, y el proceso de actualización de modelos en el gateway debe incluir la invalidación del cache como paso automático. El uso de prefijos de clave por versión de modelo (`{model_name}:{model_version}:{hash_del_prompt}`) facilita la invalidación masiva de todas las entradas de una versión específica.

Aspectos técnicos del caching en LLM Gateway

- **Exact match cache:** hash SHA256 del (model_name + system_prompt + user_message + temperatura + max_tokens) como clave de Redis; TTL configurable por modelo y por tipo de contenido; cero riesgo de respuestas incorrectas; hit rate bajo para prompts variables.
- **Semantic cache:** embedding del prompt con modelo liviano; búsqueda de nearest neighbor con threshold de similitud; retorna respuesta cacheada si similarity > threshold (empezar en 0.98, calibrar con datos reales); requiere base de datos vectorial (Redis con RedisSearch, Weaviate, Pinecone).
- **Cache key design:** incluir en la clave todos los parámetros que afectan la respuesta (model, temperature, max_tokens, system_prompt hash, user_message); excluir metadatos que no afectan la respuesta (request_id, timestamp, user_id del llamante).
- **Invalidación por versión de modelo:** al actualizar un modelo en el gateway, invalidar automáticamente todas las entradas de caché asociadas a esa versión usando prefijos de clave por versión; `DEL llm-cache:{old_model_version}:*` como operación de cleanup.
- **Cache analytics:** medir hit rate global y por equipo, distribución de TTL efectivo de las entradas, y savings estimados (tokens evitados × precio/token); exponer estas métricas en el dashboard de FinOps de la plataforma para justificar la inversión en el semantic cache.

Nota del Arquitecto: *El threshold de similitud del semantic cache debe calibrarse con datos reales de producción, no con ejemplos teóricos. El proceso correcto es: habilitar el semantic cache en modo shadow (calcular si habría habido hit, pero servir siempre el modelo real), revisar manualmente los pares (prompt_cacheado, prompt_nuevo) que habrían disparado un hit, y ajustar el threshold hasta que el porcentaje de pares con respuestas intercambiables sea >99%. Solo entonces habilitar el cache en modo real.*

El caching efectivo en el LLM Gateway puede reducir el costo de inferencia de forma significativa para las aplicaciones correctas, pero requiere diseño cuidadoso de la estrategia de invalidación y calibración del threshold de similitud para evitar retornar respuestas incorrectas. La siguiente sección cierra el ciclo de control del gateway examinando la auditoría centralizada: el sistema que registra cada llamada a un modelo con suficiente contexto para investigar incidentes, atribuir costos, y demostrar compliance.

Módulo 10 – Capítulo 07 – Sección 05

Auditoría centralizada: logging de todas las llamadas a modelos con metadatos de negocio

La auditoría centralizada en un LLM Gateway significa que cada request a un modelo de lenguaje es registrado con suficiente contexto para responder retroactivamente tres preguntas críticas: ¿qué se envió al modelo? (prompt, sistema, parámetros), ¿quién lo envió y en nombre de qué proyecto? (equipo, aplicación, usuario si está disponible), y ¿qué respondió el modelo y con qué consecuencias operacionales? (respuesta, tokens usados, latencia, costo, proveedor). Este registro no es solo un ejercicio de compliance: es la única fuente de verdad disponible cuando ocurre un incidente relacionado con el comportamiento de un LLM en producción.

Los casos de uso del audit log son variados y cada uno tiene requisitos ligeramente distintos. Para **debugging de comportamientos incorrectos**, el audit log debe contener el prompt exacto que produjo la respuesta problemática: "el modelo respondió algo inapropiado" solo puede investigarse si el prompt exacto que lo causó está disponible, y ese prompt solo existe en el log si el gateway lo registró de forma completa. Para **atribución de costos**, el audit log debe contener el `team_id` y `project_id` de cada request, los tokens de input y output, y el costo calculado; un job de agregación diario sobre estos campos produce los reportes de chargeback del Capítulo 09 sin esfuerzo adicional. Para **compliance y auditoría regulatoria**, el audit log debe ser inmutable y consultable retroactivamente: demostrar a auditores qué datos fueron procesados por qué modelo y cuándo requiere un log que no puede ser modificado después de la escritura.

La **privacidad de datos** en el audit log es uno de los problemas de diseño más delicados del sistema. En muchos casos, los prompts contienen PII (nombres, emails, números de identificación) o información confidencial (contratos, datos médicos, código propietario) que no debe almacenarse de forma plain-text en el log. Las estrategias no se excluyen mutuamente: almacenar el hash del prompt (para deduplicación y búsqueda exacta de un prompt específico) en lugar del prompt completo; almacenar el prompt completo pero cifrado con una clave que requiere autorización especial para descifrar; aplicar redacción automática de PII usando detectores como `presidio` o clasificadores de regex antes de escribir al log. La

política de retención de datos se aplica diferentemente según el tipo de log: metadatos (sin prompts) se retienen indefinidamente; logs completos (con prompts) se retienen por 90 días o el período mínimo requerido por compliance, con eliminación automática al expirar.

Las **queries de auditoría** más frecuentes deben estar disponibles como reportes pre-computados, no como consultas ad-hoc al sistema de logs. El equipo de finanzas necesita el costo mensual por equipo y proyecto —disponible en un dashboard de FinOps con actualización diaria. El equipo de seguridad necesita alertas de acceso inusual —disponible como regla de detección de anomalías con notificación en tiempo real. El equipo de compliance necesita el historial de uso de datos de usuarios específicos —disponible como query predefinida por `user_id` con respuesta en segundos. Solo las investigaciones forenses no anticipadas deben requerir acceso directo al sistema de logs raw.

Componentes del sistema de auditoría centralizada

- **Registro inmutable:** cada evento se escribe en append-only storage (S3 con Object Lock configurado, Kafka con retención configurable, o un sistema de audit logging como Immuta) que previene modificación o eliminación posterior al momento de escritura.
- **Metadatos de negocio:** campos obligatorios en cada log entry: `team_id`, `project_id`, `application_name`, `environment`, `model_used`, `provider`, `input_tokens`, `output_tokens`, `cost_usd`, `latency_ms`, `request_id`; sin estos campos el log es técnico pero no es operacionalmente útil.
- **Gestión de PII en logs:** redacción automática de PII con presidio o regex-based detectors antes de escribir al log; política configurable por equipo según la sensibilidad de sus datos; cifrado con KMS para logs que sí requieren el prompt completo.
- **Query de logs:** interfaz para buscar por `request_id` (debugging de incidentes), por `team_id` y rango de tiempo (reportes de uso y chargeback), y por patrones de prompt hash (identificar uso repetido de prompts problemáticos); respuesta en segundos sobre años de datos.
- **Integración con SIEM:** exportación de eventos de auditoría a sistemas de seguridad corporativos (Splunk, Elasticsearch/ELK, Microsoft Sentinel) para correlación con otros eventos de seguridad de la organización, cerrando el ciclo con los controles del Módulo 9.

Nota del Arquitecto: El audit log de un LLM Gateway es la primera línea de investigación cuando ocurre un incidente de IA. "El modelo respondió algo inapropiado" solo puede investigarse con el prompt exacto que lo causó, y ese prompt

solo existe en el log si el gateway lo registró. Un audit log incompleto —que registra el 95% de las llamadas— es equivalente a ningún audit log cuando el incidente que se investiga está en el 5% no registrado.

La auditoría centralizada cierra el ciclo de control del LLM Gateway: las llamadas se registran, los costos se atribuyen, los comportamientos problemáticos se pueden investigar retroactivamente, y los controles de seguridad del Módulo 9 tienen una pista de auditoría verificable. La sección de cierre examina cómo el LLM Gateway actúa como punto de control central de toda la arquitectura de LLMs de la organización.

Módulo 10 – Capítulo 07 – Sección 06

Cierre: el LLM Gateway es el punto de control central de la plataforma de IA

El LLM Gateway cumple en una plataforma de IA el mismo rol que un API Gateway en una arquitectura de microservicios: es el punto donde la política se aplica, el tráfico se observa, y la experiencia del desarrollador se estandariza. La analogía es precisa porque las razones de existencia de ambos son idénticas: sin un punto de control centralizado, cada componente del sistema re-implementa individualmente la misma lógica de autenticación, rate limiting, logging y fallback, con resultados inconsistentes y sin visibilidad global de lo que ocurre en el sistema.

Sin un LLM Gateway, la organización experimenta un conjunto de problemas que se vuelven más graves a medida que el número de equipos y aplicaciones que usan LLMs crece. Cada equipo gestiona individualmente sus API keys de OpenAI o Anthropic, con el riesgo de que una clave comprometida dé acceso ilimitado antes de que se detecte. Cada aplicación implementa su propia lógica de retry y fallback, con comportamientos inconsistentes cuando un proveedor se degrada. Los costos de inferencia no son atribuibles con precisión porque no hay un punto de logging centralizado. Las políticas de seguridad —detección de prompt injection, filtrado de contenido— se implementan en algunas aplicaciones pero no en todas. Con el gateway, todos estos problemas se resuelven una vez a nivel de plataforma.

La adopción del LLM Gateway requiere un cambio de configuración pequeño —usar `https://llm-gateway.internal/v1/chat/completions` en lugar de `https://api.openai.com/v1/chat/completions` — pero el beneficio concreto para los equipos de desarrollo es tangible e inmediato. Si OpenAI experimenta una outage parcial, el gateway los cambia automáticamente a Anthropic o Azure OpenAI sin que el equipo tenga que cambiar nada. Si el equipo está cerca de su límite de presupuesto mensual, reciben una notificación automática en Slack con datos granulares de qué está generando el gasto. Si se necesita auditarlos por compliance, el audit log del gateway tiene el registro completo de todas sus llamadas, sin que tuvieran que implementar ningún logging adicional. La plataforma entrega estos beneficios como consecuencia automática de usar el endpoint interno.

La inversión en construir y operar un LLM Gateway se amortiza rápidamente a escala. En organizaciones con más de diez equipos usando LLMs, la reducción de costos por caching semántico y routing optimizado puede ser del 30-70% sobre el costo sin optimización. La visibilidad de costos por equipo y proyecto, que el gateway provee automáticamente, hace posible las prácticas de FinOps del Capítulo 09 que sin ella serían imposibles de implementar. Los controles de seguridad centralizados eliminan el riesgo de que una aplicación sin medidas de defensa contra prompt injection comprometa toda la infraestructura de IA de la organización.

Principio rector

El LLM Gateway convierte el caos de múltiples equipos llamando directamente a múltiples APIs de modelos en un sistema gobernado: un punto de entrada, múltiples salidas, y visibilidad completa sobre quién gasta qué en qué modelo, con qué prompts, con qué resultados, y bajo qué controles de seguridad. El capítulo siguiente lleva la conversación desde el control operacional al control de governance: cómo las políticas que el gateway aplica en tiempo real se conectan con las políticas de government de datos y modelos que rigen qué puede entrenarse, qué puede desplegarse, y quién puede acceder a qué.

"Simplicity is prerequisite for reliability."

— Edsger W. Dijkstra, pionero de la ciencia de la computación, cuya máxima sobre la relación entre simplicidad y confiabilidad aplica directamente al diseño de capas de abstracción como el LLM Gateway: un sistema simple que hace bien una cosa es más confiable que un sistema complejo que intenta hacer muchas cosas a la vez.